Advanced Digital Design

PRESENTED BY:

Dr. Shasanka Sekhar Rout

Advanced Digital Design

Pre-requisites: Knowledge of Digital Fundamentals

Course Educational Objectives:

- ✓ To make the students familiar with the basic concept of digital circuits
- ✓ Introduction to digital design with CMOS implementation
- ✓ Study of combinational & sequential logic circuits and advance digital design

Course Outcomes:

- ❖ To understand and analyse the basic concept of digital logic circuits
- ❖ Able to design & demonstrate different combinational logic circuits & sequential logic circuits
- ❖ To understand CMOS implementation of digital design

SYLLABUS

Session 1
Lecture: Basic Digital Circuits
- Basic Digital Circuits: Introduction, Principles, Working.
- Usage / Applications.

Session 2
Lecture: Combinational Circuits
- Logic Arrays
- NAND-NAND / NOR-NOR circuits
- Encoders and Decoders

Session 3
Lecture: Arithmetic Circuits
- 2's Complement
- Half and Full adders
- RCA,CLA,CSA

Session 4 & 5
Lecture: Logic Minimization
- Introduction to Various Logic Minimization techniques
- Quine McCluskey technique
- Shannon's reduction using relay switches

Session 6
Lecture: Logical Hazards
- Hazards and glitches
- Hazard elimination

Session 7
Lecture: Logic Families
- Introduction to different Logic families with their sub-families/types.
- TTL – NAND using TTL with working, different types of TTL families with features, usage.
- ECL – NAND using ECL with working, different types of ECL families with features, usage.
- CMOS – NAND using CMOS with working, Different types of CMOS families with features, usage.
- Comparison of speed and power of different families and sub-families/types.

Session 8
Lecture: Tristate and other circuits
- Tristate
- Muller-C
- AOI (And-Or-Inverter)
- Design of Logic Gates with CMOS technology: NOT, NAND, NOR, XOR.

Session 9
Lecture: Sequential Circuits
- ❖ Sequential Circuit Principles
- ❖ Types of Sequential Circuits
- ❖ Clock jitter, clock skew, timing parameters, propagation delays
- ❖ Latches
- ❖ Flip-flops

Session 10 & 11
Lecture: Counter Design and Frequency Division
- ❖ Counters: Introduction and types
- ❖ Designing of Counters using FSM approach: MOD-N, Binary, Decade
- ❖ Binary Counter as a Frequency divider
- ❖ Frequency division hazards

Session 12
Lecture: Registers
- ❖ Registers
- ❖ Shift Registers
- ❖ LIFO
- ❖ FIFO
- ❖ LFSR

Session 13 & 14
Lecture: Finite State Machines (FSM)
- ❖ Types of FSM
- ❖ Problem solving
- ❖ State reduction techniques

Session 15 & 16
Lecture: Static Timing Analysis
- ❖ Need of Static Timing Analysis of Digital Circuits
- ❖ Metastability
- ❖ Setup and Hold Violations
- ❖ Time Borrowing and Time Stealing

Session 17 & 18
Lecture: Asynchronous Circuits
- ❖ Introduction to multiple clock domain circuits
- ❖ Working across multiple clock domains
- ❖ Handshaking

Session 19 & 20
Lecture: Asynchronous to Synchronous Circuit Interaction

Session 21 & 24
Lecture: Case studies of Advanced Digital Design Circuits
- ❖ Several case studies like complex FSMs, Real life problems and their solutions, Pipelined Approaches for Communication Protocols or Recursive Operations, Design Optimization, Latency reduction, Development of Microporcessor / Microcontroller Design, Circuit Development for Data Encryption / Decryption Algorithms etc.

REFERENCES

Text Book:

1. Digital Design - Principles and Practices by John F Wakerly

Reference Books:

1. An Engineering Approach to Digital Design by Fletcher

Session 1

Lecture: Basic Digital Circuits

- ✓ Basic Digital Circuits: Introduction, Principles, Working
- ✓ Usage / Applications

Basic Digital Circuits

Introduction:

- ✓ Digital circuits are the foundation of modern electronics, used in computers, communication devices, and embedded systems.
- ✓ Digital circuits operate using binary signals (0s and 1s) and are built using logic gates that perform basic operations.

- ➢ Basic Components of Digital Circuits are:
  - ✓ Logic Gates (Building blocks of digital circuits)
  - ✓ Combinational Circuits
  - ✓ Sequential Circuits

Inputs	AND	OR	NOT (A)	NAND	NOR	XOR	XNOR
0 0	0	0	1	1	1	0	1
0 1	0	1	0	1	0	1	0
1 0	0	1	1	1	0	1	0
1 1	1	1	0	0	0	0	1

- ➢ Digital circuits depends upon:
  - ✓ Binary System and Logic Levels
  - ✓ Number Systems Used in Digital Circuits
  - ✓ Boolean Algebra and Simplification

Basic Digital Circuits

Digital Circuit Design Process:

1. Problem Definition – Identify the requirements
2. Truth Table Creation – Define inputs and outputs
3. Boolean Expression – Write the logic equation
4. Simplification – Minimize the logic equation
5. Circuit Implementation – Use logic gates to build the circuit
6. Testing & Simulation – Verify the design

Technologies Used in Digital Circuits:

- **TTL (Transistor-Transistor Logic)** – Uses bipolar junction transistors (BJTs).
- **CMOS (Complementary Metal-Oxide-Semiconductor)** – Uses field-effect transistors (FETs) for low power consumption.
- **PLDs (Programmable Logic Devices)** – Include FPGA and CPLD for flexible digital design

Basic Digital Circuits

Principles & Working of Basic Digital Circuits:

- Digital circuits operate using binary signals (0s and 1s) and are based on fundamental principles that ensure reliable computation and processing.
- These principles form the backbone of digital electronics used in computers, communication devices, and embedded systems.
  - ✓ 0 represents LOW (0V or ground)
  - ✓ 1 represents HIGH (e.g., 5V, 3.3V, or 1.8V, depending on technology)
  - ❖ Logic gates perform fundamental operations on binary inputs to produce a binary output.
  - ❖ Basic Laws of Boolean Algebra:

Identity Law: $A + 0 = A, A \cdot 1 = A$

Null Law: $A + 1 = 1, A \cdot 0 = 0$

Idempotent Law: $A + A = A, A \cdot A = A$

Complement Law: $A + A' = 1, A \cdot A' = 0$

De Morgan's Theorems:

$(A \cdot B)' = A' + B'$

$(A + B)' = A' \cdot B'$

Basic Digital Circuits

Principle of Combinational and Sequential Logic:

a) Combinational Circuits

- Output depends only on the present inputs.
- Examples: **Adders, Subtractors, Multiplexers, Decoders, Encoders.**

b) Sequential Circuits

- Output depends on both present inputs and past states (memory).
- Uses Flip-Flops (FFs) to store information.
- Examples: **Counters, Registers, Finite State Machines (FSMs).**

Synchronization and Timing:

- ❖ Digital circuits rely on **clock signals** to synchronize operations.
- ❖ Sequential circuits use **clock pulses** to transition between states.
- ❖ **Propagation delay** affects circuit speed and performance.
- ➢ The working of digital circuits is based on binary logic, logic gates, Boolean algebra, combinational/sequential logic, and clock signals.
- ➢ These circuits process, store, and transmit data efficiently, forming the backbone of modern computing and electronic devices.

Basic Digital Circuits

Usage / Applications:

Digital circuits are everywhere like control systems in cars, communication networks, and even medical equipment, which are the backbone of modern electronics.

1. Computing and Microprocessors:

- **Microprocessors & Microcontrollers**– The heart of all computers, smartphones.
- **Memory Units (RAM, ROM, Flash Storage)**– Used for data storage.
- **Arithmetic Logic Units (ALU)**– Mathematical and logical operations in CPUs.
- **Registers and Caches**– Store and process temporary data in processors.

Example: Laptop's CPU processes instructions using millions of digital logic gates.

2. Communication Systems

- **Digital Signal Processing (DSP)**– Used in audio, video, and wireless.
- **Modems and Network Devices**– Convert digital data into signals for transmission.
- **Error Detection & Correction Circuits**– Ensure data integrity in networks.
- **Encryption and Security Circuits**– Secure data transfer in communication systems.

Example: Wi-Fi router and 5G network use digital circuits for fast and secure data transmission.

Basic Digital Circuits

3. Consumer Electronics:

- **Smartphones & Tablets**– Use microprocessors, memory, and display drivers.
- **Television & Displays**– Digital logic controls image processing.
- **Cameras & Image Processing**– Convert analog signals (light) into digital data.
- **Audio Processing (Speakers, Headphones, Music Systems)**– Uses DSP to enhance sound quality.

Example: Noise-canceling headphones use digital circuits to analyze and reduce unwanted sounds.

4. Industrial Automation and Robotics:

- **Programmable Logic Controllers (PLCs)**– Control industrial machinery.
- **Robotic Control Systems**– Process sensor data and control motors.
- **Digital Sensors & Actuators**– Used for precision in manufacturing.
- **Embedded Systems in IoT Devices**– Collect and process data for automation.

Example: Factory automation robots use digital circuits to follow programmed instructions.

Basic Digital Circuits

5. Medical Electronics:

- **Digital Thermometers & Wearable Devices**– Convert temperature readings into digital values.
- **Heart Rate Monitors & ECG Machines**– Process signals from the body into digital form.
- **MRI & CT Scanners**– Use digital processing for imaging.
- **Hearing Aids & Pacemakers**– Use digital logic for real-time processing.

Example: A smartwatch tracks heart rate using digital circuits.

6. Automotive Electronics:

- **Engine Control Units (ECU)**– Manage fuel injection, emissions, and performance.
- **Automatic Transmission Systems**– Process sensor data to shift gears.
- **Anti-lock Braking System (ABS)**– Uses digital circuits for safety.
- **Infotainment Systems & Navigation (GPS)**– Convert digital data for display and control.

Example: Your car's dashboard screen and parking sensors rely on digital circuits.

Basic Digital Circuits

7. Aerospace and Defense:

- **Radar & Satellite Communication**– Uses digital signal processing for tracking and communication.
- **Autopilot Systems in Aircraft**– Process sensor data for navigation.
- **Military Encryption Systems**– Secure communication using digital circuits.
- **Missile Guidance & Defense Systems**– Digital processors analyze and track targets.

Example: A fighter jet's radar system relies on digital circuits for precise tracking.

8. Smart Home & Internet of Things (IoT):

- **Home Automation (Smart Lights, AC, Locks)**– Controlled by digital circuits.
- **Digital Assistants (Alexa, Google Home, Siri)**– Process voice commands using digital circuits.
- **Surveillance & Security Systems**– Digital cameras and sensors detect motion.
- **Wearable IoT Devices**– Track health and fitness data.

Example: A smart thermostat adjusts your home temperature using digital circuits.

Session 2

Lecture: Combinational Circuits

- ✓ Logic Arrays
- ✓ NAND-NAND / NOR-NOR circuits
- ✓ Encoders and Decoders

Combinational Circuits

- Logic circuits for digital systems may be combinational or sequential.
- A combinational circuit consists of logic gates whose outputs at any time are determined from only the present combination of inputs.
- In contrast, sequential circuits employ storage elements in addition to logic gates that means the outputs depends on present inputs and past outputs.

- For n input variables, there are 2^n possible combinations of the binary inputs.
- Example of combinational circuit: Adders, Subtractors, Logic arrays, MUX, DEMUX, Encoders, Decoders etc.

Combinational Circuits

Logic Arrays:

- A Logic Array is a structured arrangement of logic gates designed to perform multiple logical or arithmetic operations simultaneously.
- They are commonly used in digital systems for arithmetic processing, data routing, and logic functions.
- They follow a predefined architecture that allows for scalability and efficient design.

Types of Logic Arrays:

1. Programmable Logic Array (PLA)
2. Programmable Array Logic (PAL)
3. Field Programmable Gate Array (FPGA)
4. Gate Arrays (ASICs and Standard Cell Designs)

Combinational Circuits

1. Programmable Logic Array (PLA):

- A **PLA** is a configurable combinational circuit where both the AND and OR gates are programmable. It allows for **custom logic function implementation** in a single chip.

Structure:

- Input variables go into an **AND gate array**, which generates product terms.
- The outputs of the AND gate array feed into an **OR gate array** to produce the sum of products for the required Boolean functions.
- The output of the OR gate goes to an XOR gate, where the other input can be programmed to receive a signal equal to either logic 1 or logic 0.
- The output is inverted when the XOR input is connected to 1 (since $x \oplus 1 = x'$).
- The output does not change when the XOR input is connected to 0 (since $x \oplus 0 = x$).

Combinational Circuits

Advantages:

- Highly flexible (both AND and OR planes are programmable).
- Can implement multiple logic functions in a compact form.

Example: Used in custom digital logic design for memory addressing and arithmetic circuits.

Problem: Implement the following two Boolean functions with a PLA:

$$F_1 = AB' + AC + A'BC'$$

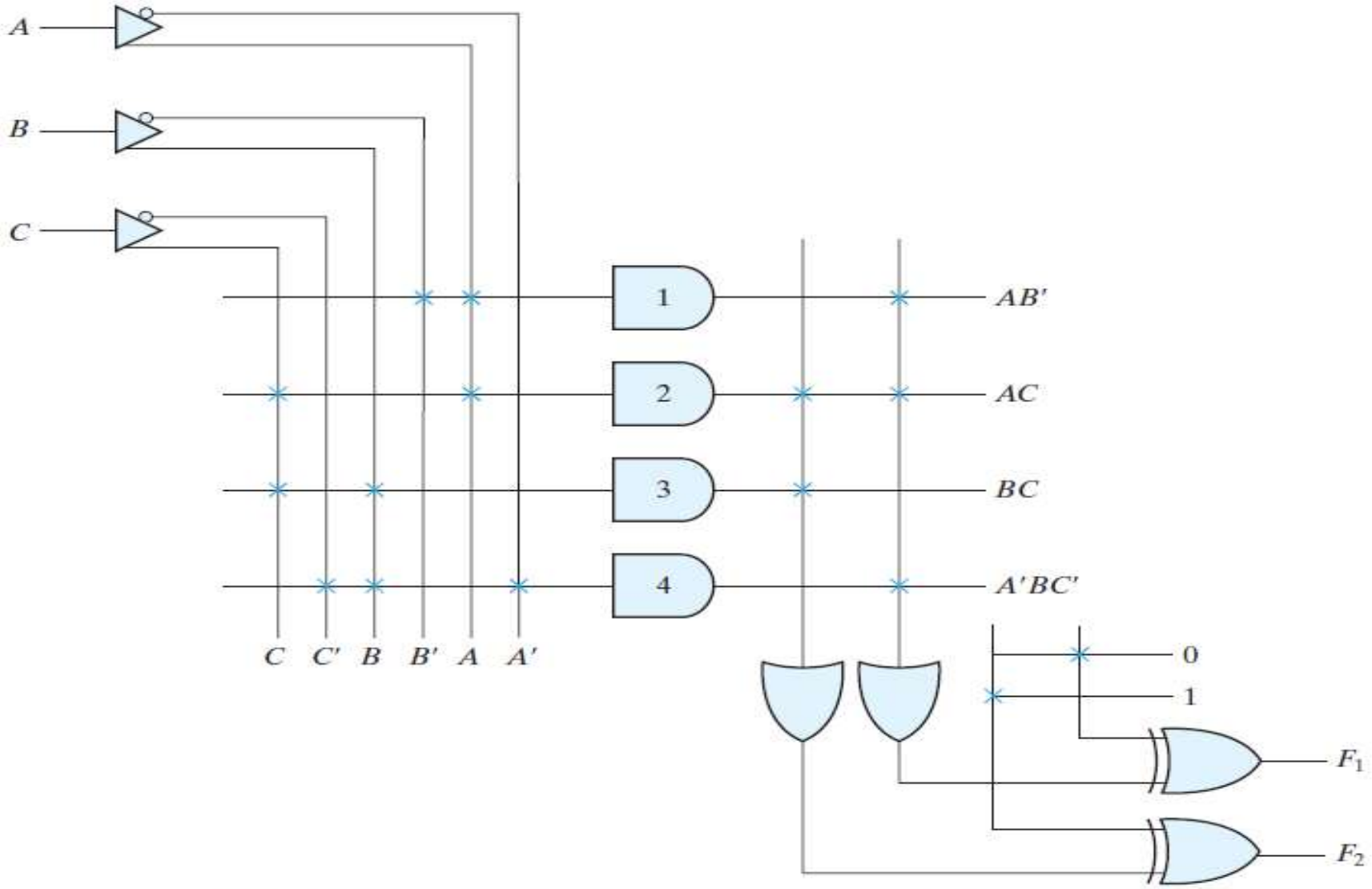
$$F_2 = (AC + BC)'$$

Solution:

PLA Programming Table

		Inputs			Outputs (T)	(C)
	Product Term	A	B	C	F_1	F_2
AB'	1	1	0	—	1	—
AC	2	1	—	1	1	1
BC	3	—	1	1	—	1
$A'BC'$	4	0	1	0	1	—

Combinational Circuits

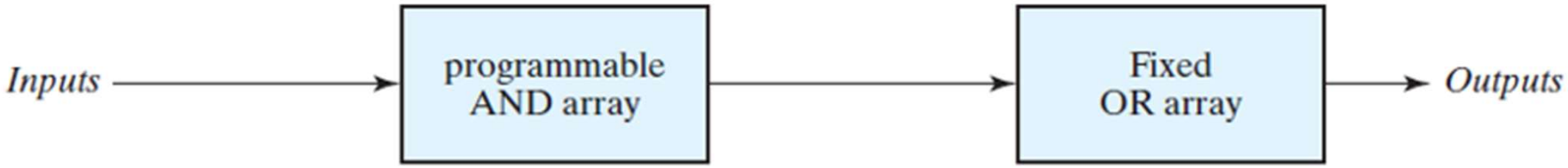


PLA with three inputs, four product terms, and two outputs

Combinational Circuits

2. Programmable Array Logic (PAL):

- A **PAL** is similar to a PLA but with a fixed OR gate array and a programmable AND gate array. It is easier to manufacture and use than a PLA.



Advantages:

- Faster than PLA due to fixed OR gates.
- Simpler and cost-effective.

Example: Used in control systems and microcontroller interfacing.

✓ Because only the AND gates are programmable, the PAL is easier to program than, but is not as flexible as, the PLA.

Combinational Circuits

Problem: Implement the following Boolean functions with a PAL:

$$w = ABC' + A'B'CD'$$

$$x = A + BCD$$

$$y = A'B + CD + B'D'$$

$$z = ABC' + A'B'CD' + AC'D' + A'B'C'D$$

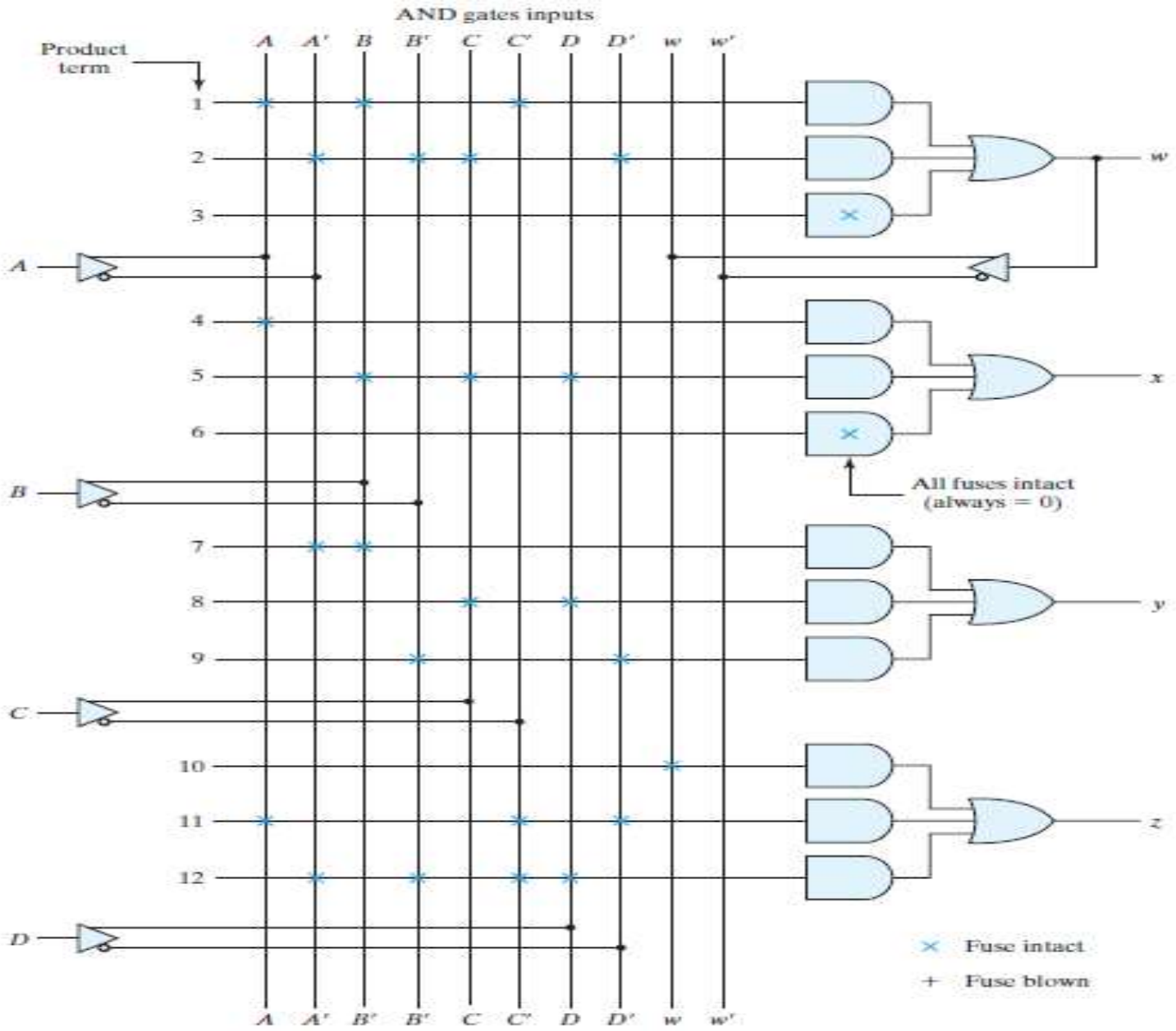
$$= w + AC'D' + A'B'C'D$$

Solution:

PAL Programming Table

Product Term	AND Inputs A	B	C	D	w	Outputs
1	1	1	0	—	—	$w = ABC' + A'B'CD'$
2	0	0	1	0	—	
3	—	—	—	—	—	
4	1	—	—	—	—	$x = A + BCD$
5	—	1	1	1	—	
6	—	—	—	—	—	
7	0	1	—	—	—	$y = A'B + CD + B'D'$
8	—	—	1	1	—	
9	—	0	—	0	—	
10	—	—	—	—	1	$z = w + AC'D' + A'B'C'D$
11	1	—	0	0	—	
12	0	0	0	1	—	

Combinational Circuits



PAL with four inputs, four outputs, and four-wide AND–OR structure

Combinational Circuits

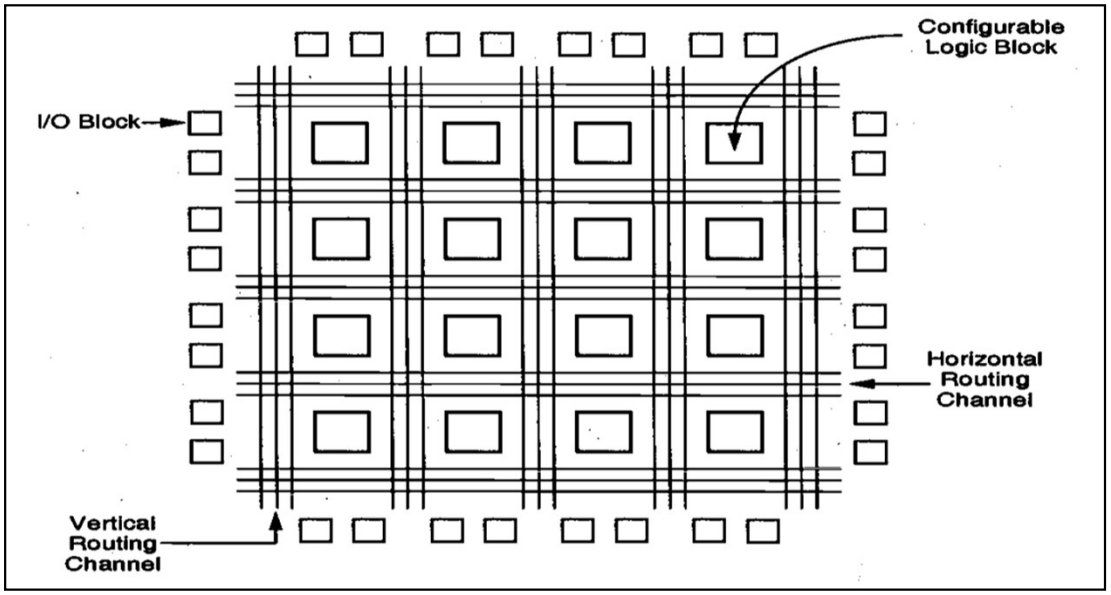
3. Field Programmable Gate Array (FPGA):

- An **FPGA** is an advanced form of logic array where both combinational and sequential circuits can be implemented using programmable logic blocks.

Advantages:

- Highly reconfigurable and reusable.
- Used in prototyping and hardware acceleration.

Example: Used in AI, image processing, networking, and real-time processing applications.



Combinational Circuits

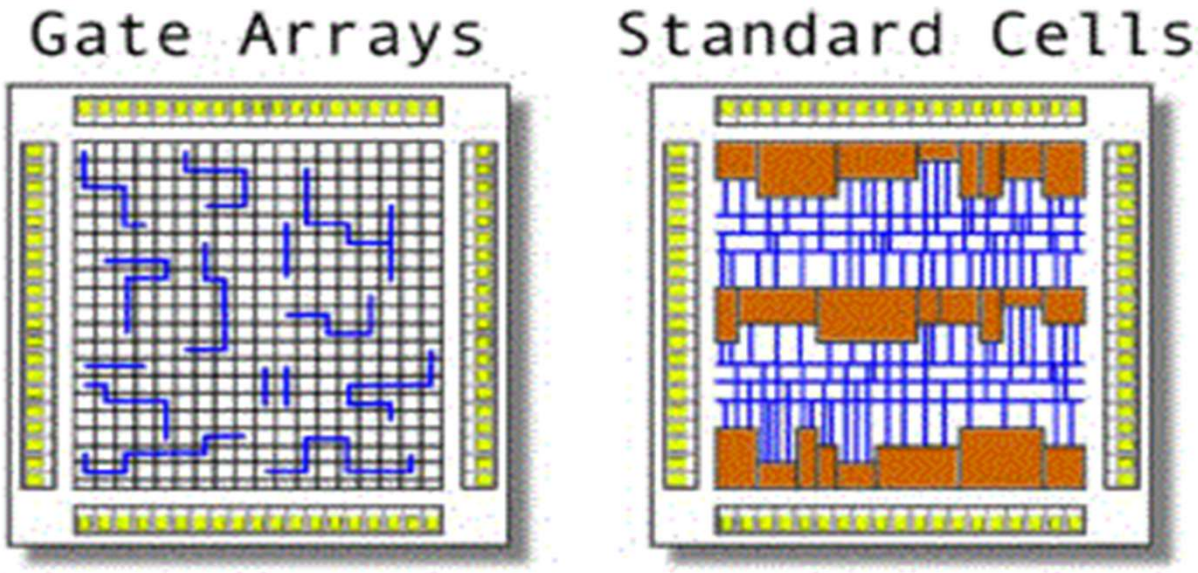
4. Gate Arrays (ASICs and Standard Cell Designs):

- Gate arrays are pre-designed logic cells that are connected based on specific design requirements. These are used in **Application-Specific Integrated Circuits (ASICs)** for high-performance and mass production.

Advantages:

- Optimized for performance and power efficiency.
- Ideal for high-volume production (e.g., mobile processors, GPUs).

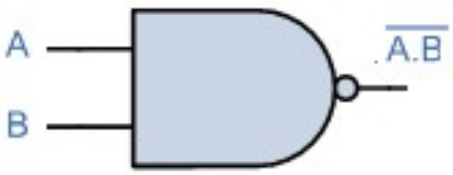
Example: Used in smartphones, gaming consoles, and automotive electronics.



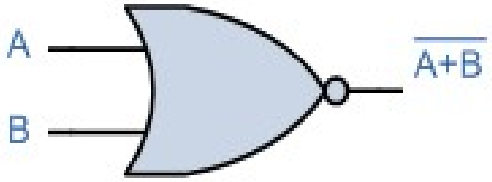
Combinational Circuits

NAND-NAND / NOR-NOR circuits:

- ❖ NAND and NOR gates are **Universal gates**, meaning they can be used to design any logic gates and implement any Boolean function or logic circuit.
- ❖ By structuring digital circuits using only **NAND** or **NOR** gates, we achieve cost-effective, efficient, and reliable designs in digital systems.
- ✓ A **NAND-NAND circuit** is a combinational logic circuit where logic functions are implemented using only **NAND gates**.
- ✓ A **NOR-NOR circuit** follows the same principle but only uses **NOR gates** to implement logic functions



INPUT		OUTPUT
A	B	Q
0	0	1
0	1	1
1	0	1
1	1	0

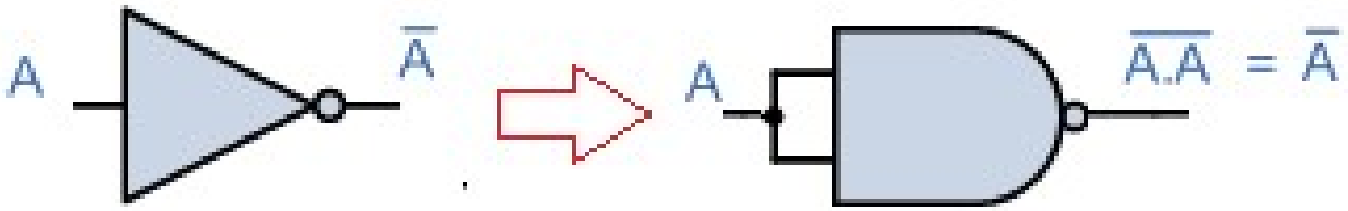


INPUT		OUTPUT
A	B	Q
0	0	1
0	1	0
1	0	0
1	1	0

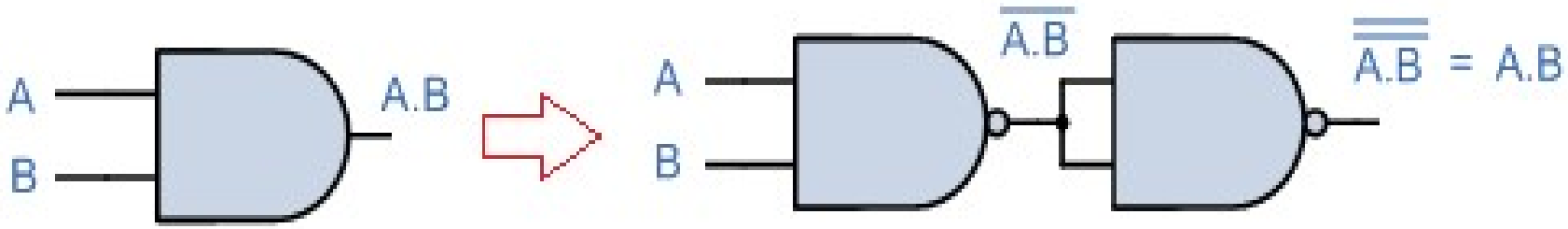
Combinational Circuits

NAND-NAND circuits:

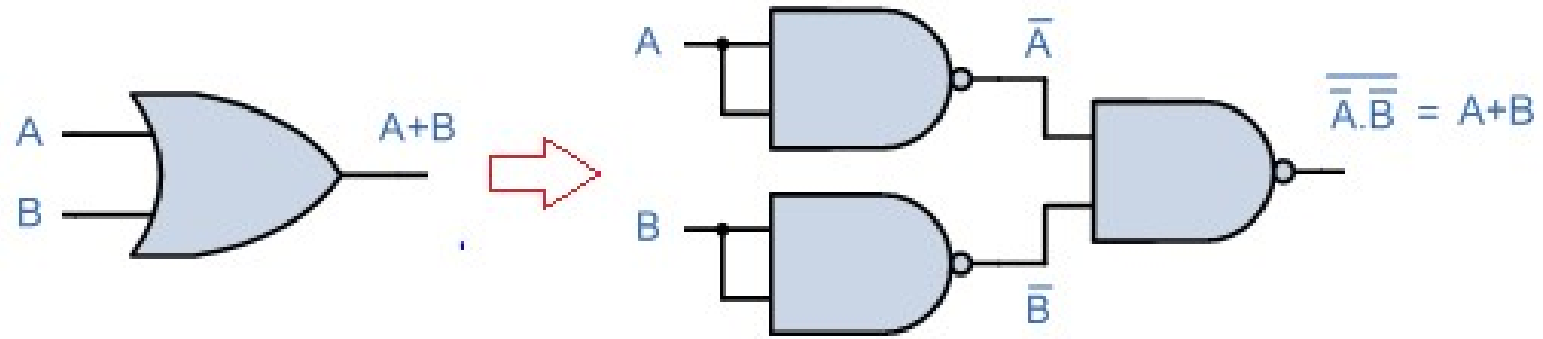
NOT:



AND:



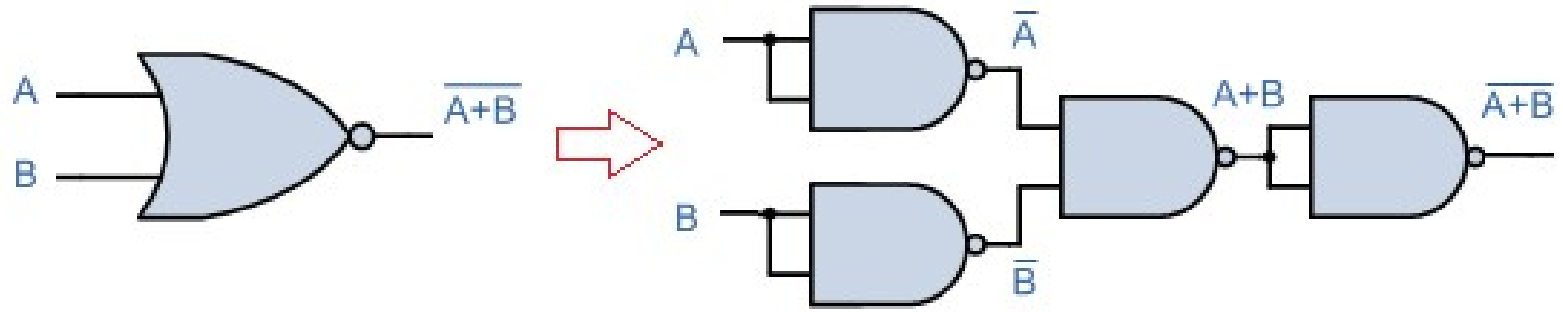
OR:



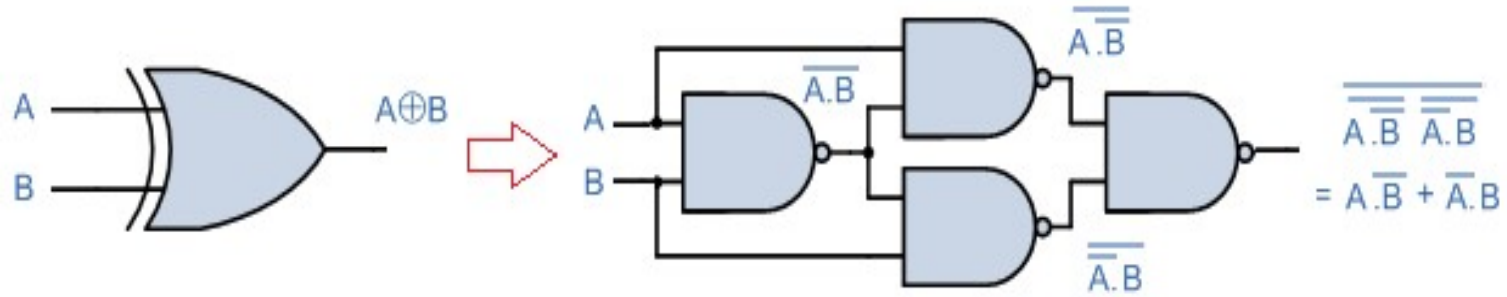
Combinational Circuits

NAND-NAND circuits:

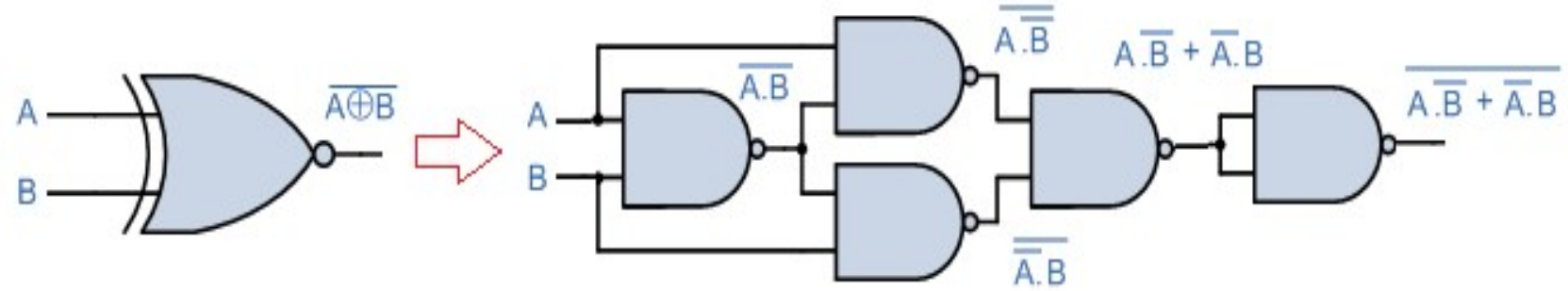
NOR:



EXOR:



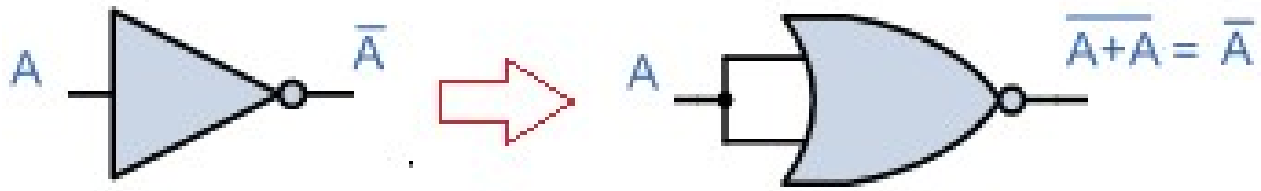
EXNOR:



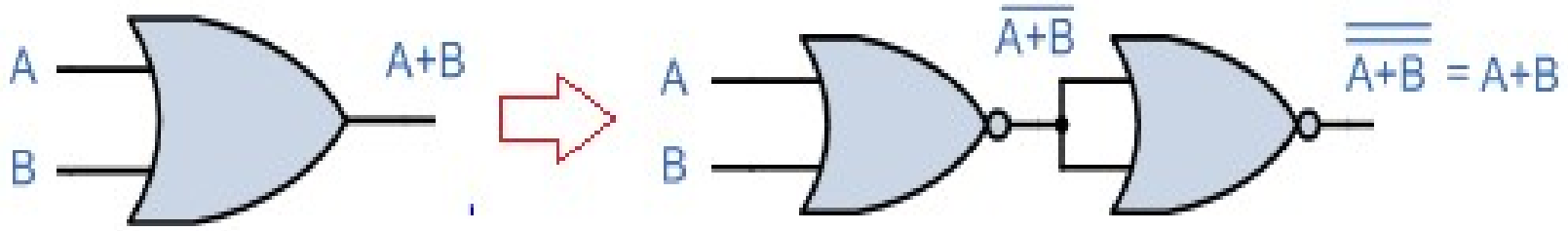
Combinational Circuits

NOR-NOR circuits:

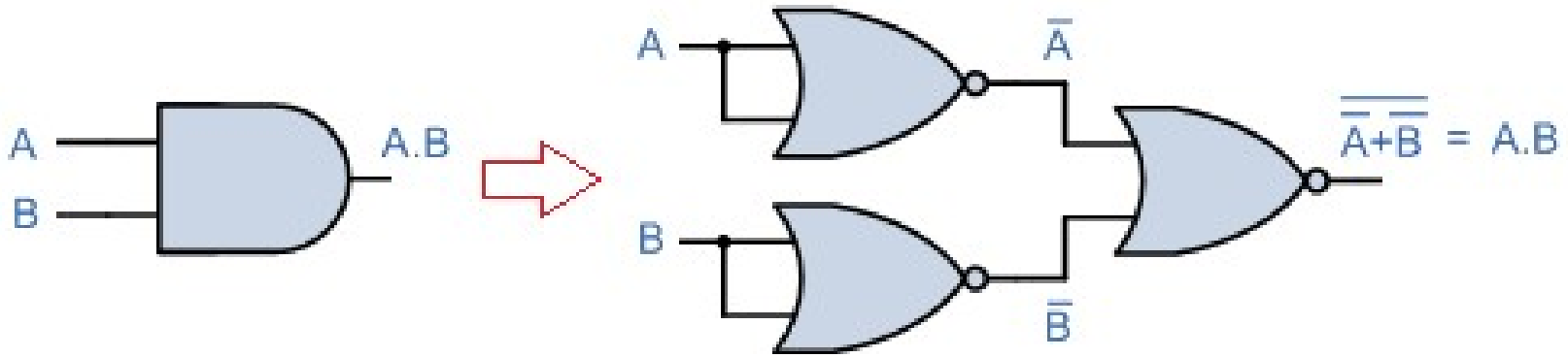
NOT:



OR:



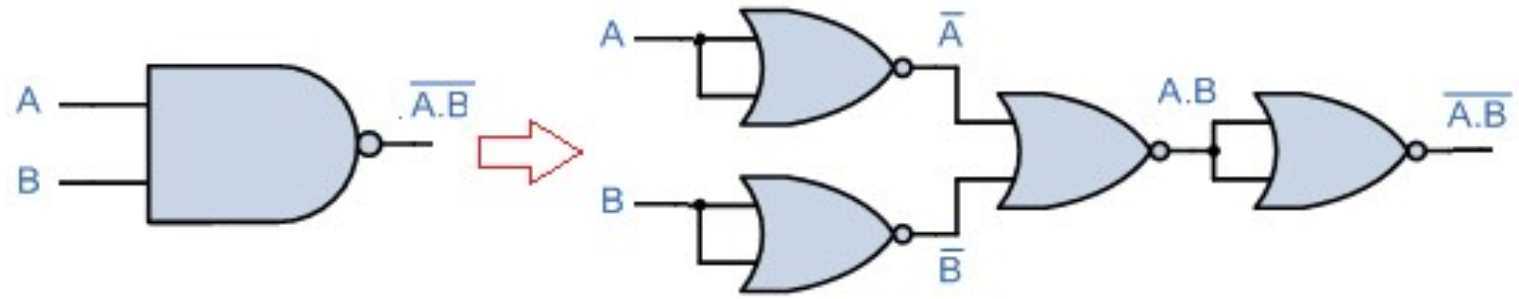
AND:



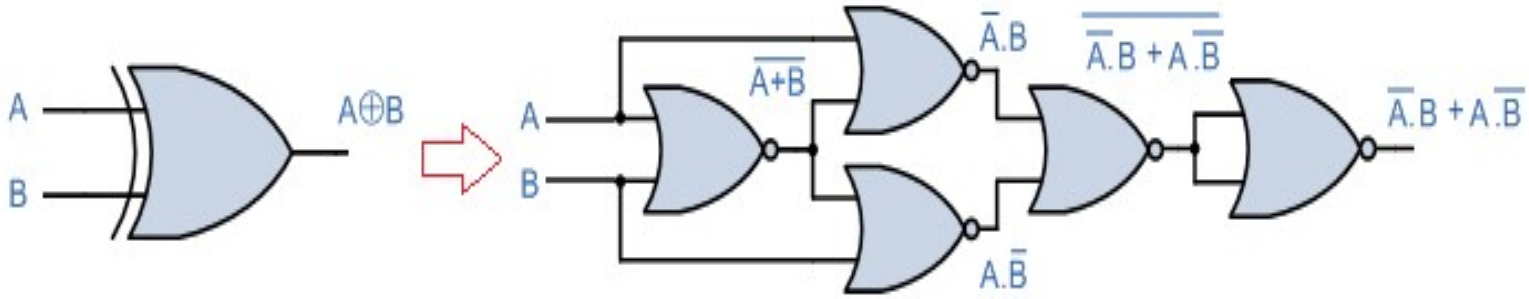
Combinational Circuits

NOR-NOR circuits:

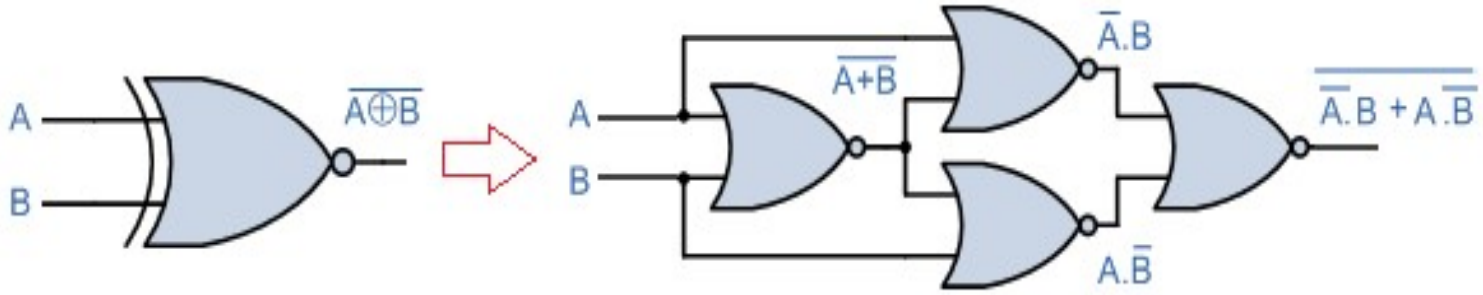
NAND:



EXOR:



EXNOR:

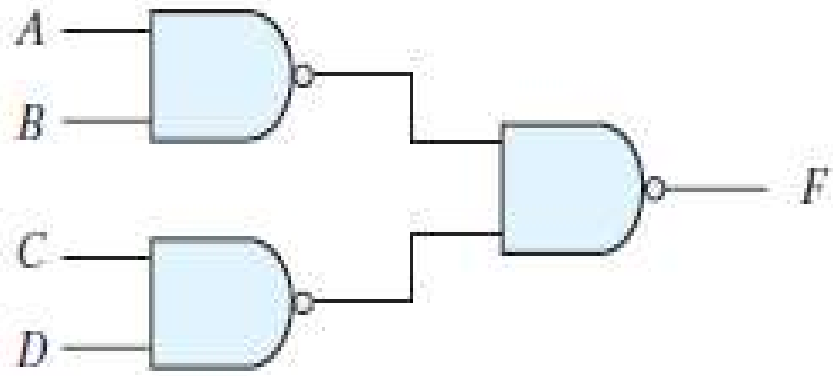


Combinational Circuits

NAND & NOR Implementation:

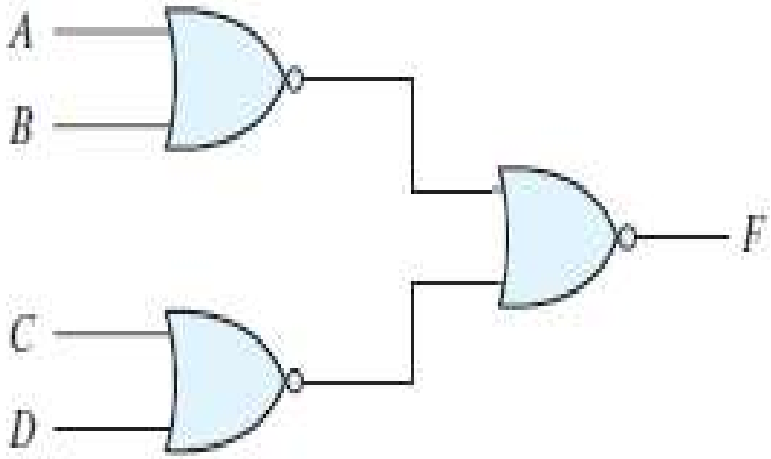
Ex: F = AB + CD

$$F = AB + CD$$
$$= \overline{\overline{AB + CD}}$$
$$= \overline{\overline{AB} \cdot \overline{CD}}$$



Ex: F = (A + B)(C + D)

$$F = (A + B).(C + D)$$
$$= \overline{\overline{(A + B).(C + D)}}$$
$$= \overline{\overline{(A + B)} + \overline{(C + D)}}$$



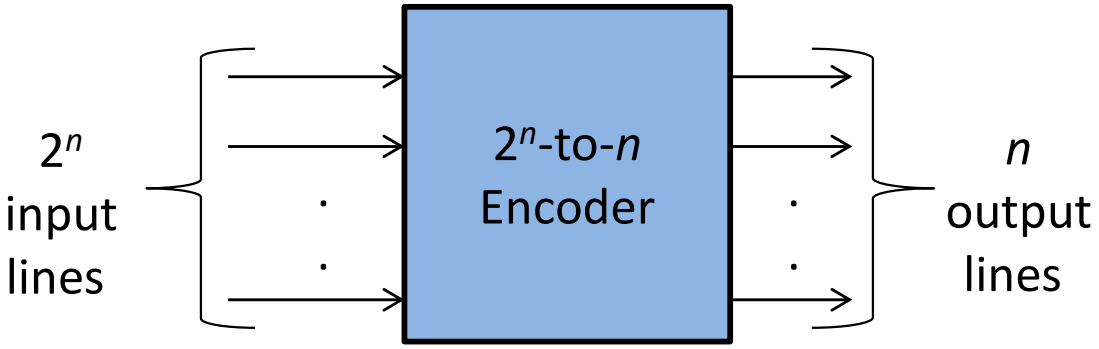
Combinational Circuits

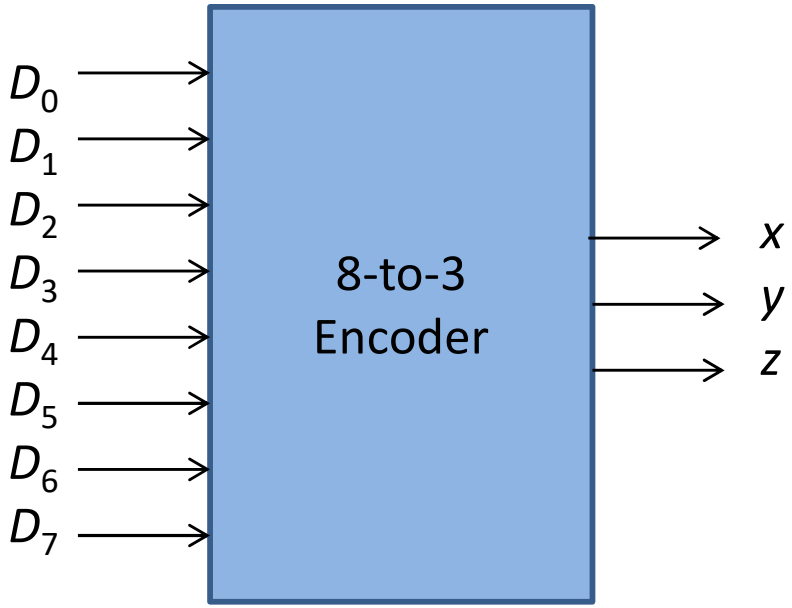
Encoders and Decoders:

Encoders and decoders are essential combinational circuits used for **data conversion, signal transmission, and control applications** in digital electronics.

Encoders:

- An encoder is a digital circuit that performs the inverse operation of a decoder.
- An encoder has 2^n (or fewer) input lines and n output lines.





Combinational Circuits

Encoders (8-to-3):

Truth Table of an Octal-to-Binary Encoder

Inputs								Outputs		
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

These conditions can be expressed by the following Boolean output functions:

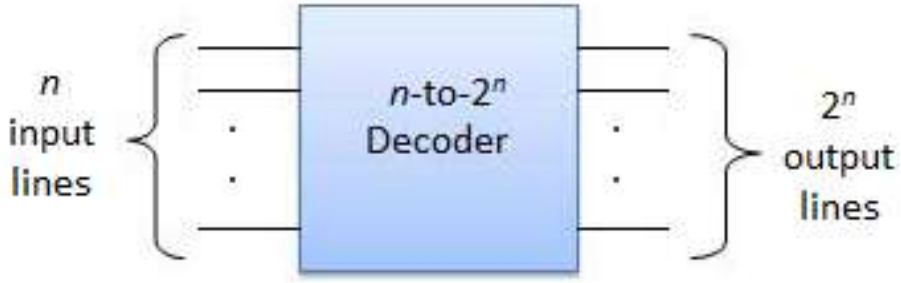
$$z = D_1 + D_3 + D_5 + D_7$$
$$y = D_2 + D_3 + D_6 + D_7$$
$$x = D_4 + D_5 + D_6 + D_7$$

The encoder can be implemented with three OR gates.

Combinational Circuits

Decoders:

❖ A decoder is a combinational circuit that converts binary information from n input lines to a maximum of 2^n unique output lines.

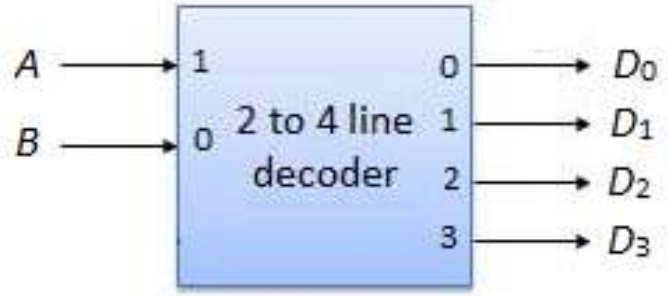


❖ The decoders presented here are called n-to-m-line decoders, where $m \leq 2^n$.

2-to-4-Line Decoder:

In 2-to-4-line decoder circuit, the two inputs are decoded into four outputs, each representing one of the minterms of the two input variables.

The two inputs are represented by A and B and outputs are represented by D_0, D_1, D_2 and D_3.



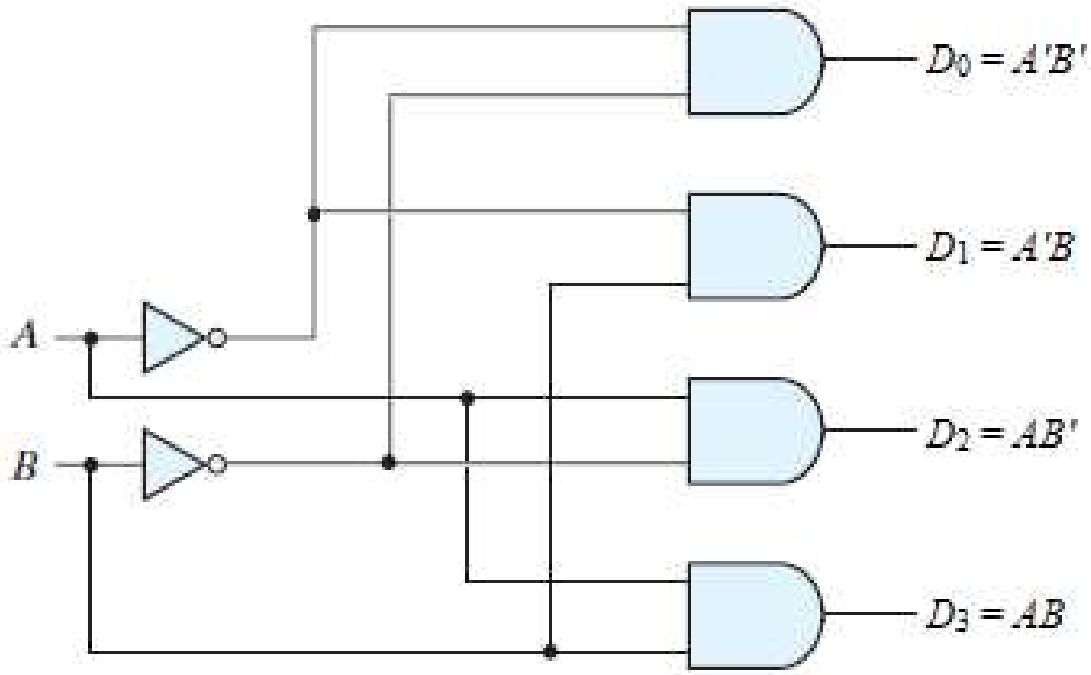
Block diagram

Combinational Circuits

2-to-4-Line Decoder:

Inputs		Outputs			
A	B	D_0	D_1	D_2	D_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

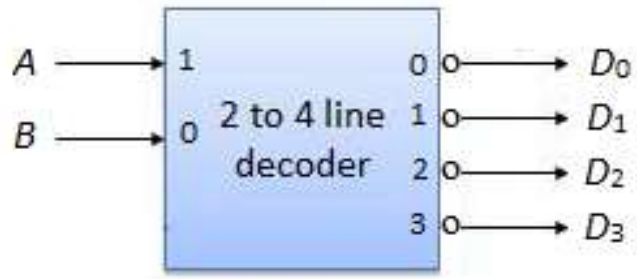
Truth table



Logic diagram

Combinational Circuits

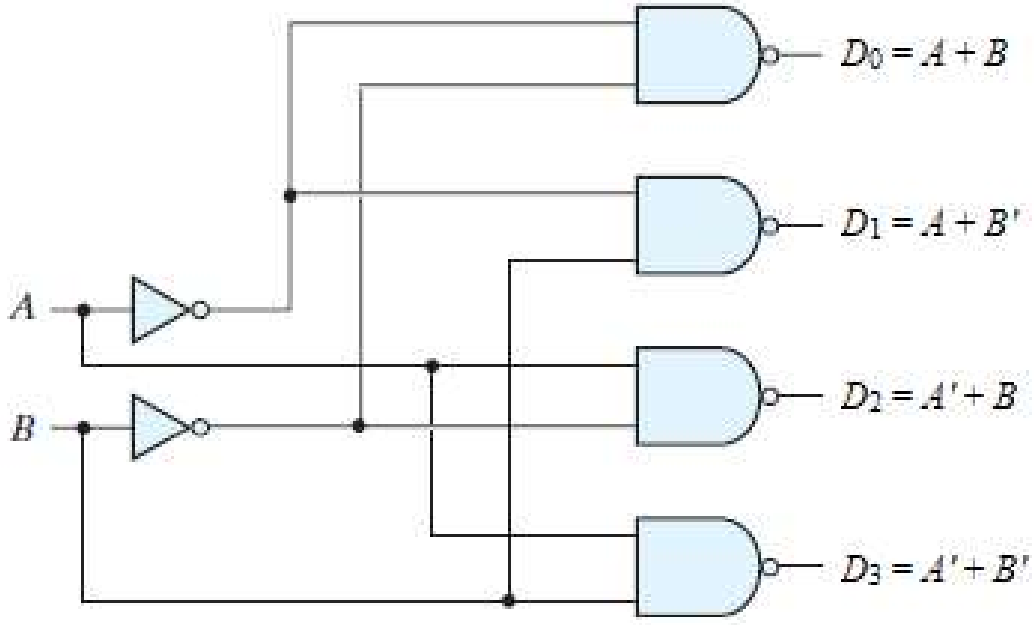
2-to-4-Line Active Low Decoder:



Block diagram

Inputs		Outputs			
A	B	D_0	D_1	D_2	D_3
0	0	0	1	1	1
0	1	1	0	1	1
1	0	1	1	0	1
1	1	1	1	1	0

Truth table

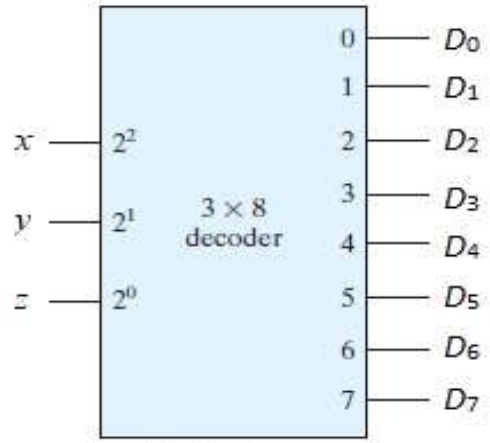


Logic diagram

Combinational Circuits

3-to-8-Line Decoder:

In 3-to-8-line decoder circuit, the three inputs are decoded into eight outputs, each representing one of the minterms of the three input variables.



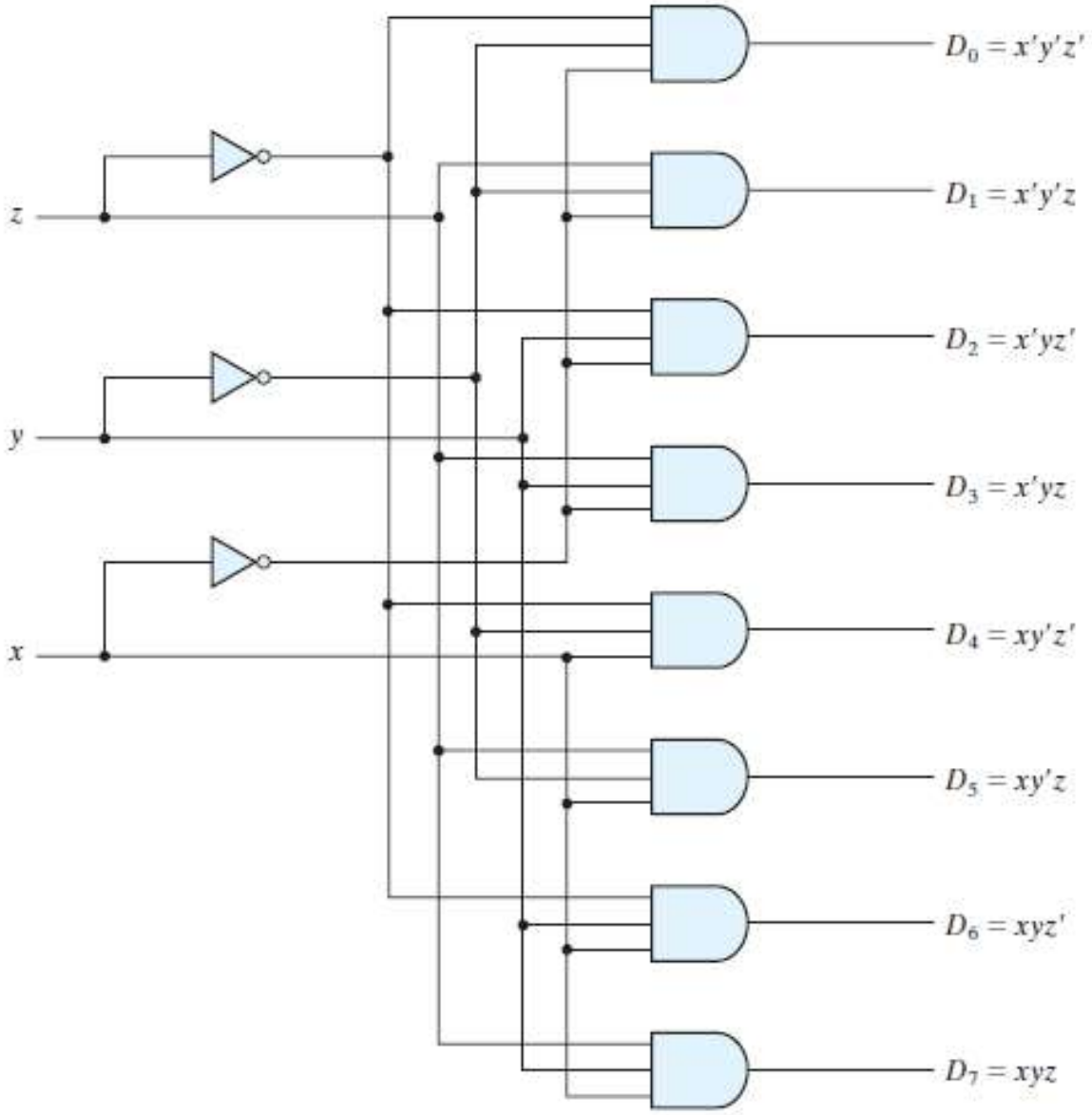
Block diagram

Truth Table of a Three-to-Eight-Line Decoder

Inputs			Outputs							
x	y	z	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Combinational Circuits

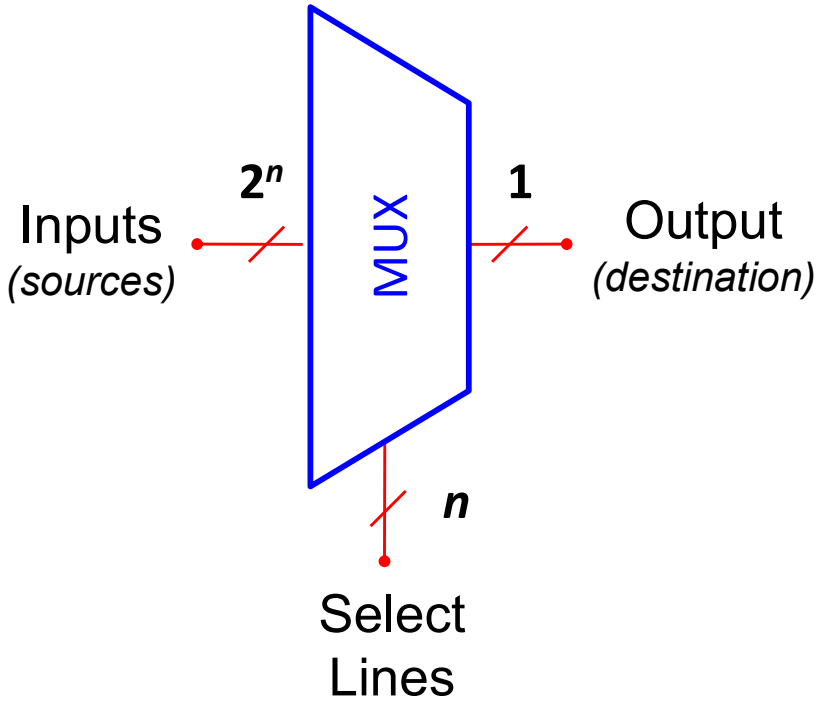
3-to-8-Line Decoder:

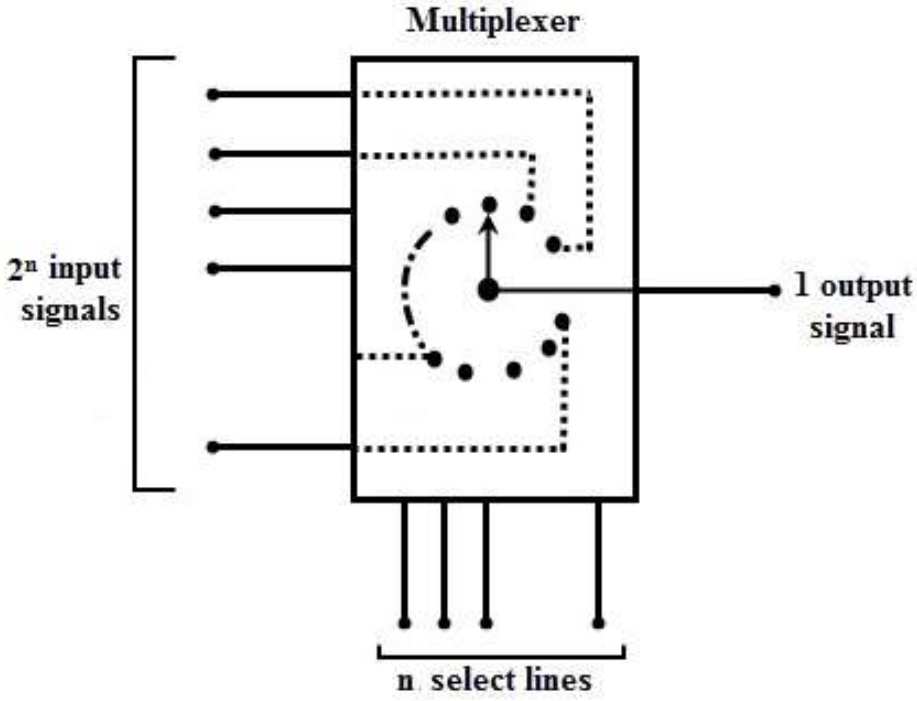


Three-to-eight-line decoder

Multiplexers:

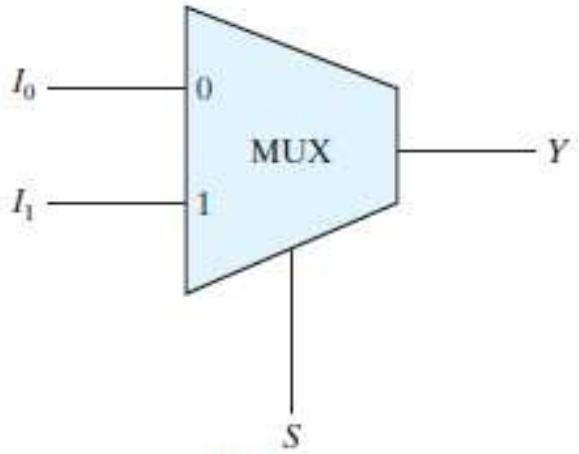
- A multiplexer is a combinational circuit that selects binary information from one of many input lines and directs it to a single output line.
- The selection of a particular input line is controlled by a set of selection lines.
- Normally, there are 2^n input lines and n selection lines whose bit combinations determine which input is selected.
- A multiplexer is also called a data selector, since it selects one of many inputs and steers the binary information to the output line.





Two-to-One-line Multiplexer:

- A two-to-one-line multiplexer connects one of two 1-bit sources to a common destination, as shown in Fig.
- The circuit has two data input lines, one output line, and one selection line *S*.

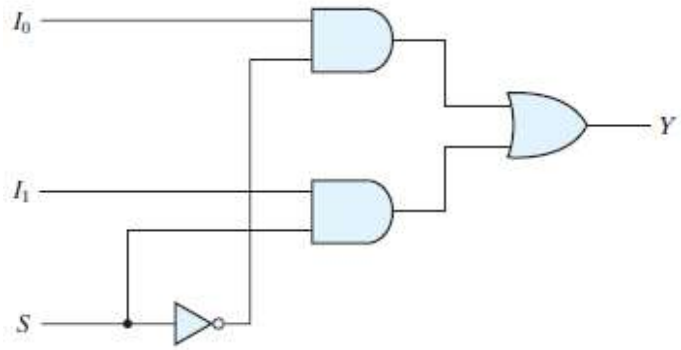


Block diagram

S	Y
0	I_0
1	I_1

Function table

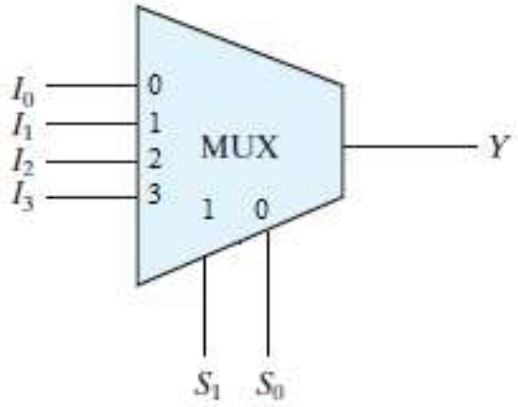
$$Y = I_0\overline{S} + I_1 S$$



Logic diagram

Four-to-One-line Multiplexer:

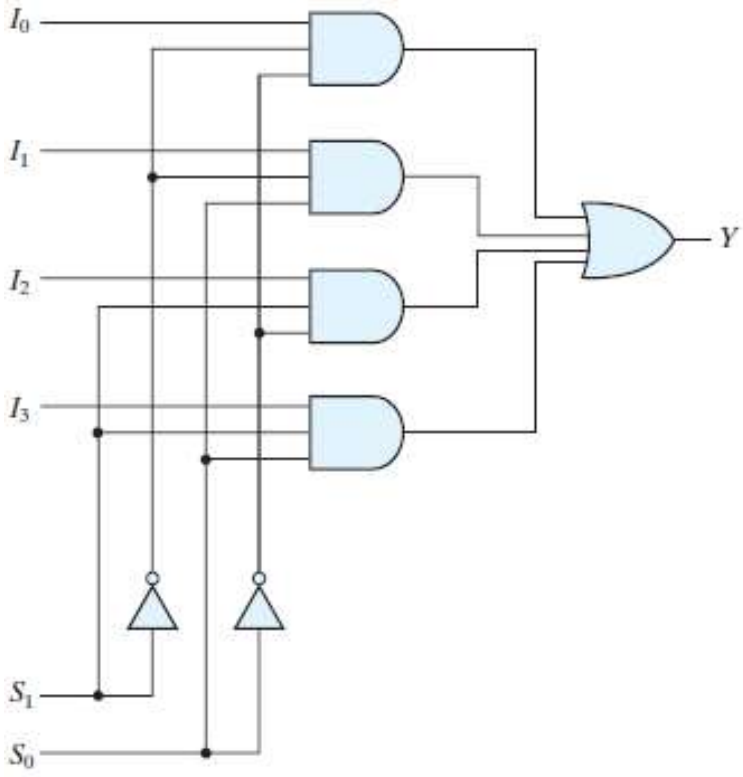
- Each of the four inputs, I_0 through I_3, is applied to one input of an AND gate.
- Selection lines S_1 and S_0 are decoded to select a particular AND gate.
- The outputs of the AND gates are applied to a single OR gate that provides the one-line output.



Block diagram

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

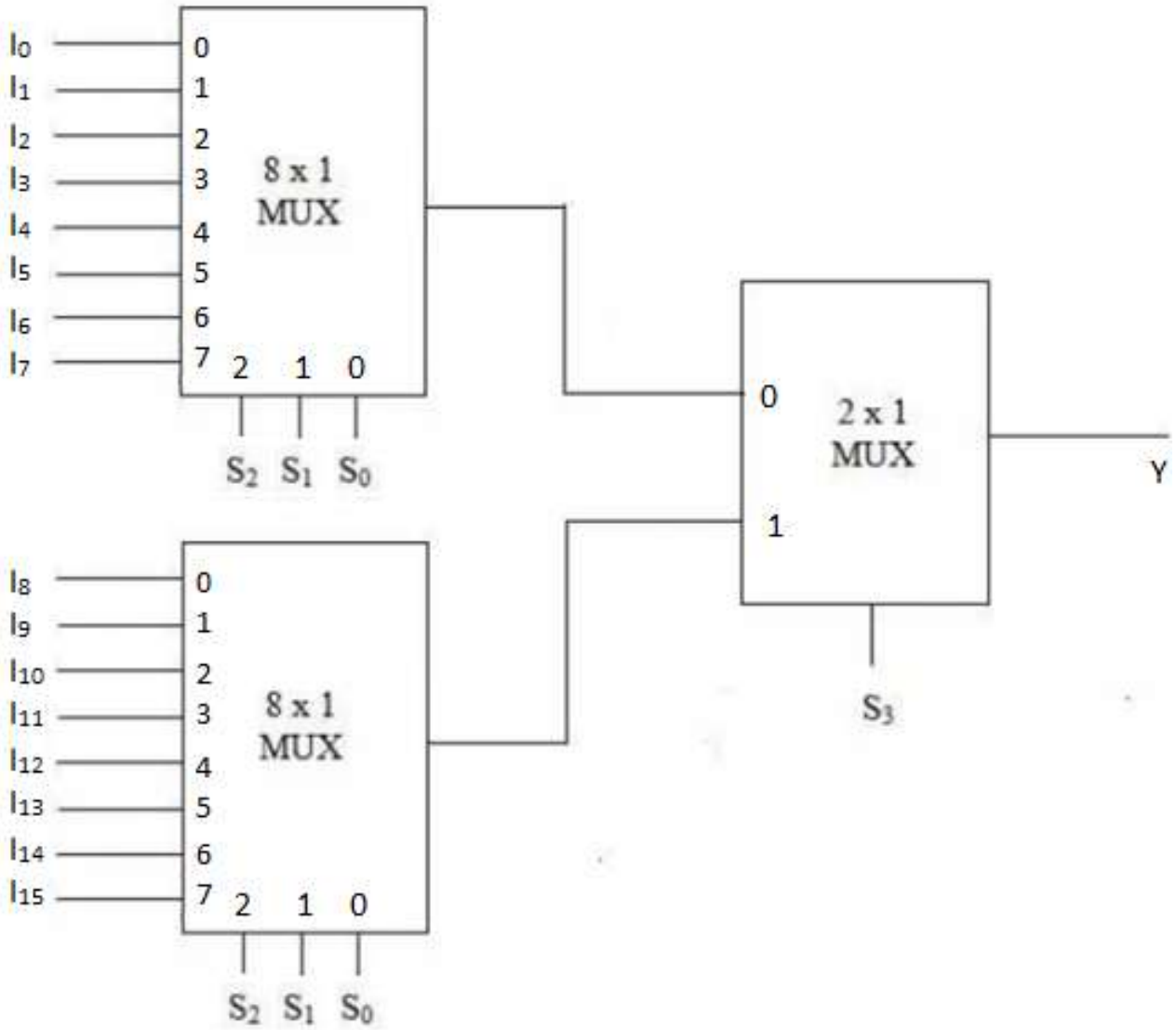
Function table



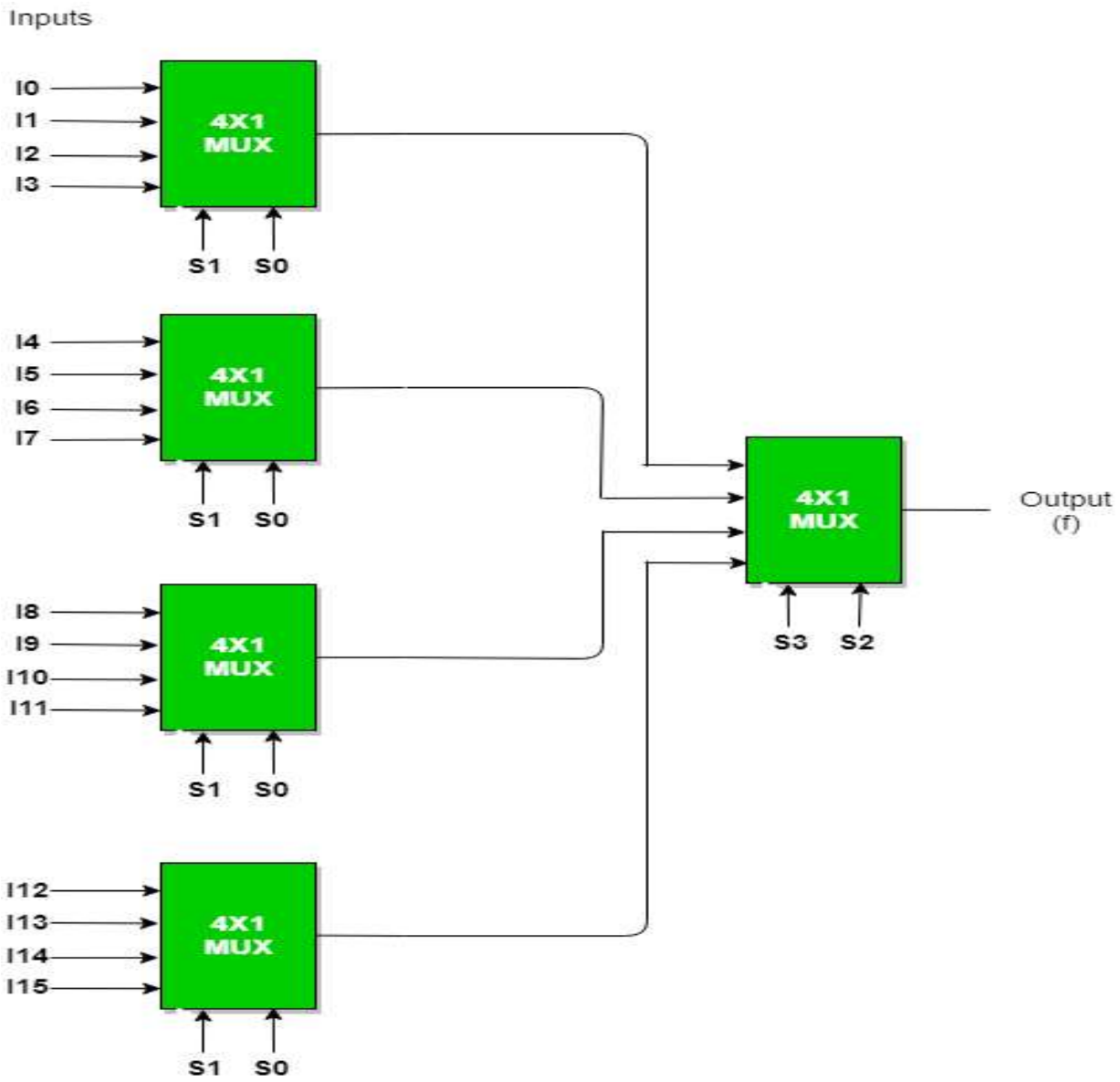
Logic diagram

$$Y = I_0\overline{S}_1\overline{S}_0 + I_1\overline{S}_1S_0 + I_2S_1\overline{S}_0 + I_3S_1S_0$$

Example: Construct a 16 × 1 multiplexer with two 8 × 1 and one 2 × 1 multiplexers.

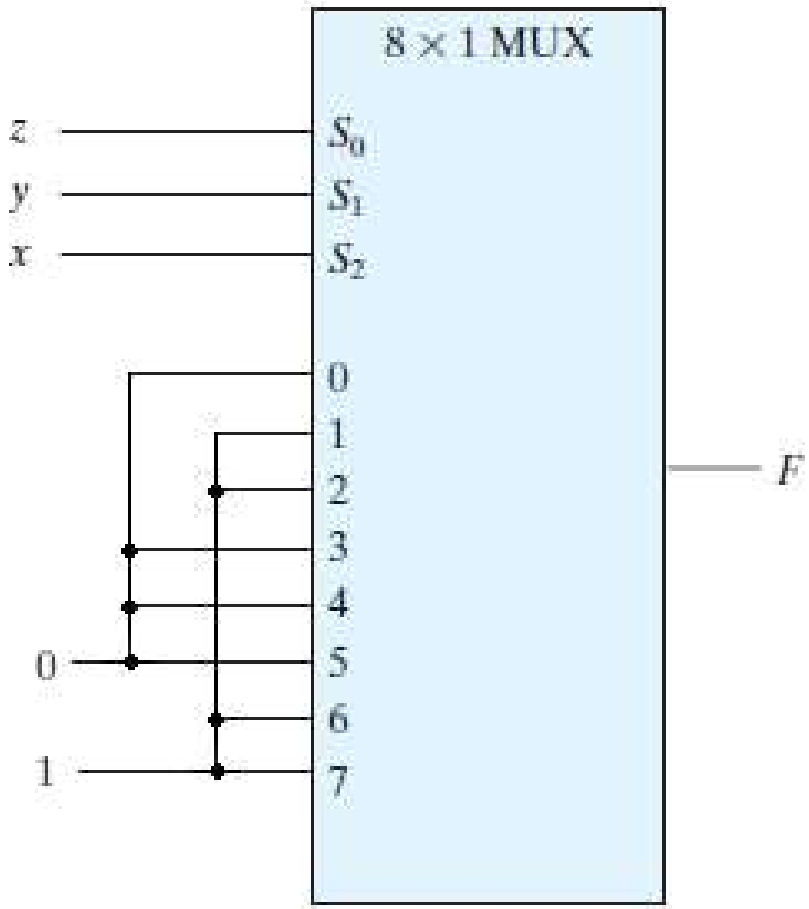


Example: Construct a 16 × 1 multiplexer with five 4 × 1 multiplexers.



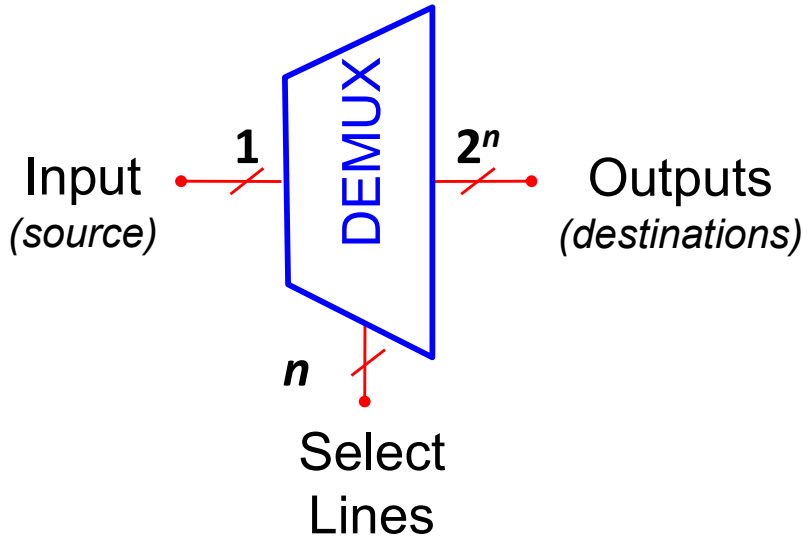
Example: Implement the following Boolean function with a multiplexer.

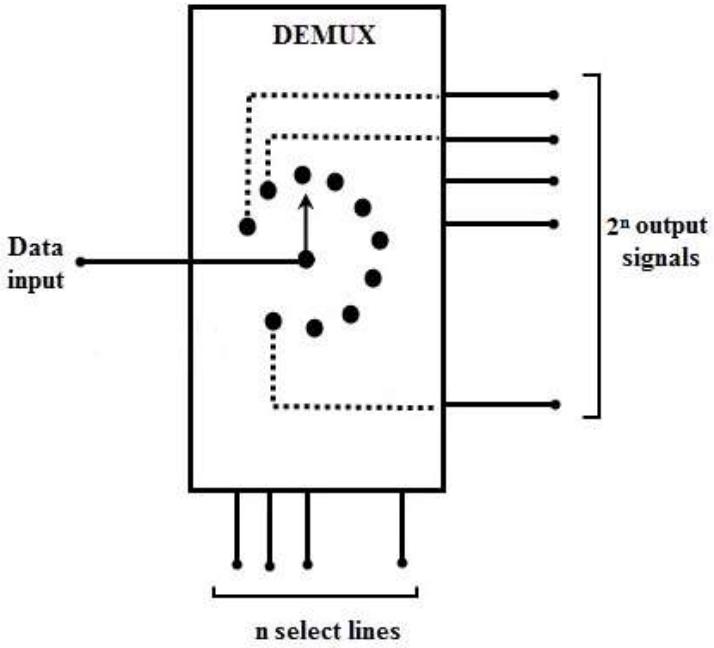
$F(x, y, z) = \sum(1, 2, 6, 7)$



De-multiplexers:

- A de-multiplexer performs the reverse operation of multiplexer; it takes single input and distributes it over several outputs.
- A decoder with enable input can function as a de-multiplexer - a circuit that receives information from a single line and directs it to one of 2^n possible output lines.
- The selection of a specific output is controlled by the bit combination of n selection lines.
- A de-multiplexer is also called a data distributor, since it transmits the same data to different destinations.

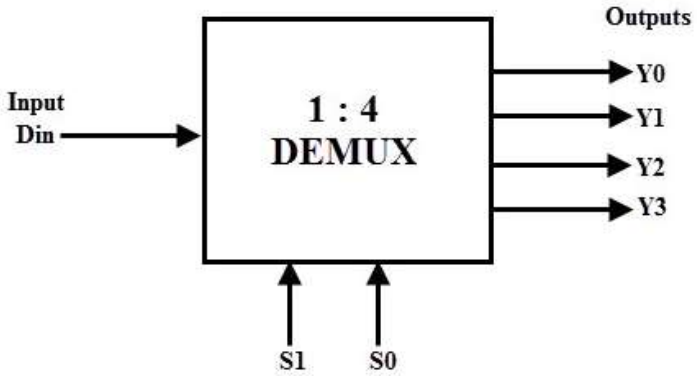




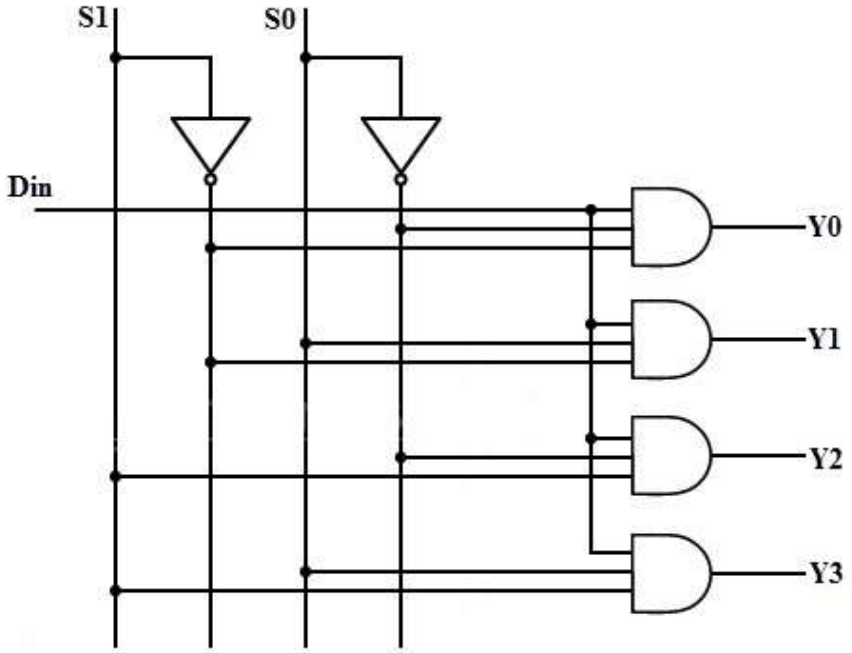
1-to-4 Line De-multiplexer:

- A 1-to-4 line de-multiplexer has a single input (D), two selection lines (S_1 and S_0) and four outputs (Y_0 to Y_3).
- The input data goes to any one of the four outputs at a given time for a particular combination of select lines.

$$Y_0 = \overline{S}_1 \overline{S}_0 D$$
$$Y_1 = \overline{S}_1 S_0 D$$
$$Y_2 = S_1 \overline{S}_0 D$$
$$Y_3 = S_1 S_0 D$$

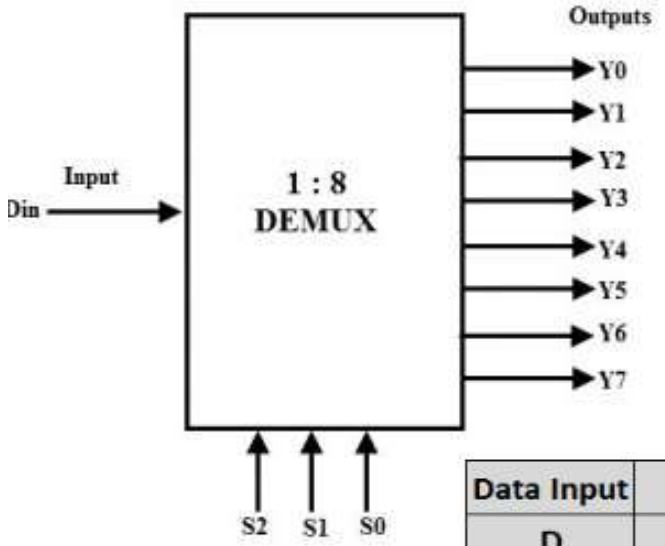


Data Input	Select Inputs		Outputs			
D	S_1	S_0	Y_3	Y_2	Y_1	Y_0
D	0	0	0	0	0	D
D	0	1	0	0	D	0
D	1	0	0	D	0	0
D	1	1	D	0	0	0



1-to-8 Line De-multiplexer:

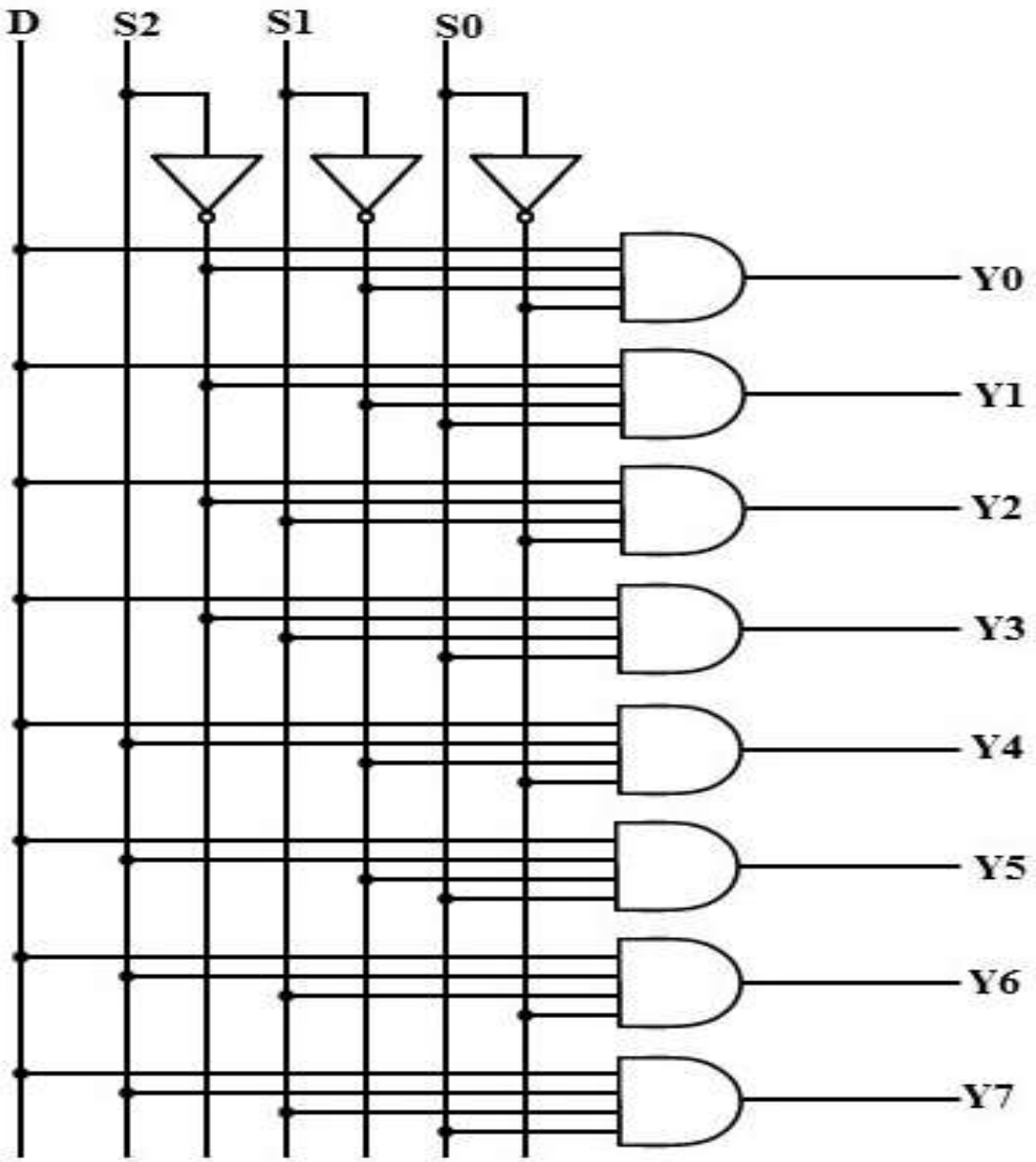
- A 1-to-8 de-multiplexer consists of single input D, three select inputs S_2, S_1 and S_0 and eight outputs from Y_0 to Y_7.
- It distributes one input line to one of 8 output lines depending on the combination of select inputs.



$$Y_0 = \overline{S}_2\overline{S}_1\overline{S}_0D \qquad Y_4 = S_2\overline{S}_1\overline{S}_0D$$
$$Y_1 = \overline{S}_2\overline{S}_1S_0D \qquad Y_5 = S_2\overline{S}_1S_0D$$
$$Y_2 = \overline{S}_2S_1\overline{S}_0D \qquad Y_6 = S_2S_1\overline{S}_0D$$
$$Y_3 = \overline{S}_2S_1S_0D \qquad Y_7 = S_2S_1S_0D$$

Data Input	Select Inputs			Outputs							
D	S_2	S_1	S_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
D	0	0	0	0	0	0	0	0	0	0	D
D	0	0	1	0	0	0	0	0	0	D	0
D	0	1	0	0	0	0	0	0	D	0	0
D	0	1	1	0	0	0	0	D	0	0	0
D	1	0	0	0	0	0	D	0	0	0	0
D	1	0	1	0	0	D	0	0	0	0	0
D	1	1	0	0	D	0	0	0	0	0	0
D	1	1	1	D	0	0	0	0	0	0	0

1-to-8 Line De-multiplexer:



THANK YOU